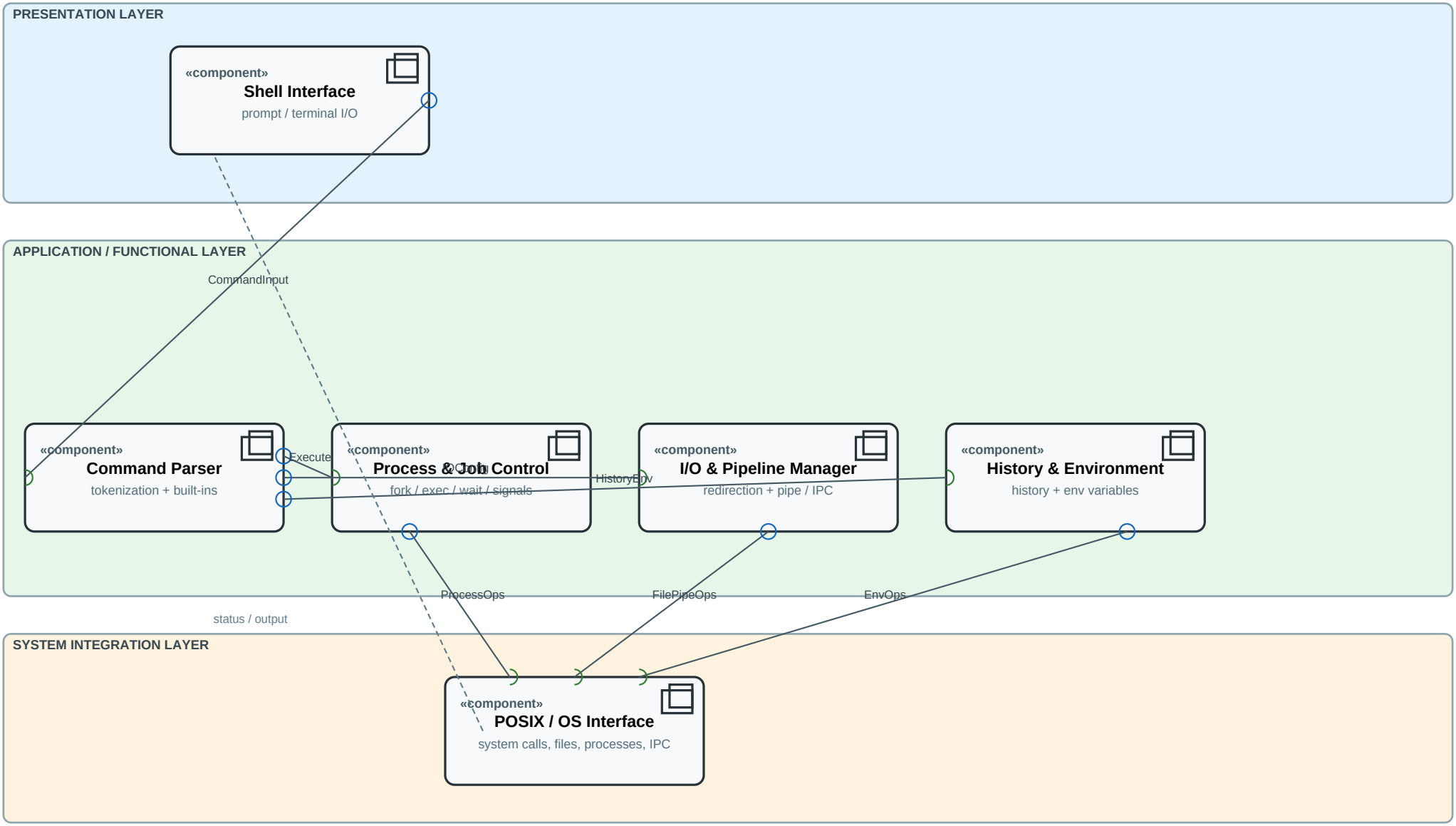
# LAB 3 — COMPONENT MODELLING & ARCHITECTURAL PATTERN SELECTION

Simple Shell — Command Line Interpreter | Selected Architecture: Layered




**UML notation:** Provided Required

Flow: input → parsing → functional services → POSIX/OS → output

# LAB 3 — ARCHITECTURAL JUSTIFICATION

Simple Shell — Command Line Interpreter

### Architecture Selection
We chose Layered Architecture for the Simple Shell system. The architecture separates the user-facing shell interface, functional modules, and lower-level POSIX/operating-system interaction.

### Reason 1 — Clear separation of shell responsibilities
The Lab 1 system is already divided into command parsing and built-ins, process execution and job control, I/O redirection and pipelines, and command history and shell environment. A layered structure keeps these responsibilities modular and makes each functional area independently testable and maintainable.

### Reason 2 — Appropriate for a local, modular C application
The shell runs locally in a Linux/Unix-like environment and is not intended to provide remote execution or a network service. Layered communication through local module interfaces avoids the operational complexity and network overhead associated with a distributed microservices design while preserving clear module boundaries.

### Security Advantage
Separating command parsing from lower-level OS operations provides a controlled path for validating commands and arguments before system-level actions are requested. This supports the Lab 1 requirements for input validation, malformed-command handling, memory-safety checks and robustness testing.

### Performance Benefit
The functional modules communicate within the same local process, so normal command handling does not require network communication between services. This fits the shell's interactive workload while allowing direct use of POSIX process, file-descriptor and IPC mechanisms.

### Component Mapping
- 1. Shell Interface — accepts commands and displays prompts/output.
- 2. Command Parser & Built-ins — tokenizes input and dispatches supported built-in commands.
- 3. Process & Job Control — manages external processes, foreground/background execution and signals.
- 4. I/O & Pipeline Manager — handles redirection and single/multiple pipelines using file descriptors and IPC.
- 5. History & Environment — manages command history and session-level environment variables.
- 6. POSIX / OS Interface — represents the underlying operating-system mechanisms used by the functional modules.